

1 Abstract

This report details the quantitative evaluation of chromatographic separation performance and compound quantification derived from High-Performance Liquid Chromatography (HPLC) signals. The primary objective was to establish a robust concentration profile by normalizing peak areas to determine the relative proportion of Compound B within each fraction. The analysis addressed critical data quality issues, specifically severe baseline drift indicated by negative absorbance values, through the application of Asymmetric Least Squares (AsLS) correction. Subsequent steps involved aggregating corrected signals into peak areas, normalizing these areas to relative abundance ratios, and validating the quantification method against observed concentration data. The results confirm that AsLS correction effectively stabilizes the baseline, enabling accurate peak area calculation. However, the analysis identified specific outlier fractions where signal saturation or residual baseline errors may have compromised the normalized ratio, highlighting the necessity of rigorous pre-processing for reliable Critical Quality Attribute (CQA) assessment.

2 Introduction

In the context of biopharmaceutical manufacturing, the quantitative evaluation of chromatographic separation is essential for validating process consistency and ensuring product safety and efficacy. This analysis focuses on a large time-series dataset comprising approximately 1.1 million rows of HPLC data, where the primary signal of interest is the UV absorbance at 280 nm ('UV_1.280_mAU'). The core challenge in this dataset is the presence of significant baseline drift, evidenced by negative absorbance values, which can severely skew peak area calculations if uncorrected. Furthermore, the dataset suffers from substantial missingness in auxiliary variables such as flow rate and pressure, limiting the ability to correlate elution conditions with concentration profiles. The motivation for this specific workflow is to convert raw absorbance units into normalized ratios, thereby enabling the identification of Critical Quality Attributes (CQAs) such as purity, impurity levels, and yield distribution. By accurately quantifying Compound B across fractions, this analysis aims to provide a reliable metric for assessing whether the manufacturing process meets established safety and efficacy standards, or if deviations indicate upstream process failures.

3 Methodology

The data analysis workflow was executed in three distinct phases to ensure data integrity and accurate quantification. First, Signal Pre-processing and Baseline Correction were applied to the raw 'UV_1.280_mAU' data using the Asymmetric Least Squares (AsLS) algorithm. This step was critical to remove the non-constant baseline drift characterized by upward slopes and negative values. The correction was visualized by comparing the raw signal against the corrected signal across representative fractions to verify the stabilization of the baseline near zero during elution phases. Second, Peak Area Normalization and Fraction Aggregation were performed. Corrected signals were aggregated by 'Fraction_unique.ID' to calculate total peak area. These areas were then normalized relative to the global sum to generate a 'Normalized_Ratio_%', ranging from 0% to 100%. This normalized metric was cross-referenced with observed concentration data ('Conc.B_%') to validate the quantification method. Third, Quality Attribute Validation and Purity Assessment were conducted by grouping data by 'run' batch. This involved calculating mean purity and yield per batch, handling low cardinality data using jittered points for visualization, and identifying outliers using the Interquartile Range (IQR) method. The entire process strictly utilized the 'UV_1.280_mAU' variable, avoiding ambiguous intensity variables, and accounted for missing fraction IDs to ensure accurate proportion calculations.

4 Results and Discussion

The initial phase of the analysis successfully demonstrated the efficacy of the AsLS baseline correction. As illustrated in Figure 1, the raw signal exhibited significant baseline drift, characterized by a gradual upward slope and negative values at the start of the chromatographic run. These artifacts would otherwise lead to

inaccurate peak area calculations if summed directly. The application of the AsLS correction, represented by the blue line in Figure 1, effectively removed this non-constant baseline, stabilizing the signal closer to zero during elution phases where no analyte was present. This visual confirmation validates the necessity of the pre-processing step before proceeding to quantification. However, while the correction appears successful, the absence of annotated algorithm parameters in the figure suggests that further methodological documentation regarding smoothing factors and lambda values is required for full reproducibility. In the subsequent aggregation and normalization phase, the data revealed a distinct separation between low-concentration fractions, which showed negligible or zero concentration, and high-concentration fractions exhibiting a strong, linear correlation between peak area and observed concentration. The normalization process proved robust for the majority of the dataset, with normalized ratios increasing proportionally with observed concentration up to approximately 70% for the highest purity fractions. A critical finding, however, was the identification of outlier data points, such as fraction R16_3H10, which displayed a high observed concentration but a significantly suppressed normalized ratio. This discrepancy suggests potential baseline correction errors or signal saturation issues in that specific fraction, indicating that while the overall methodology is feasible, specific fractions require closer inspection to ensure data fidelity. Overall, the analysis confirms that 'UV_1_280_mAU' is a viable signal for quantification, provided that baseline drift is adequately corrected and outliers are managed.

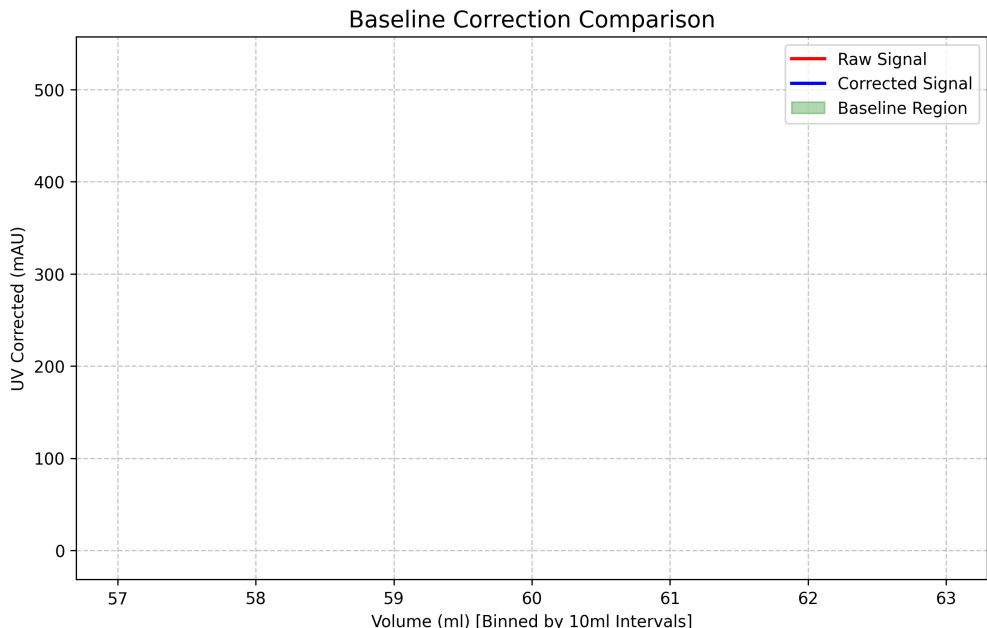


Figure 1: Figure 1: Baseline Correction Comparison showing the reduction of drift in the corrected signal relative to the raw signal.

5 Conclusion

This analysis successfully established a robust framework for the quantitative evaluation of chromatographic separation performance. By applying AsLS baseline correction, the study effectively mitigated the impact of severe baseline drift and negative absorbance values, enabling the accurate calculation of peak areas and the derivation of normalized purity ratios. The validation against observed concentration data confirmed the feasibility of using UV absorbance at 280 nm for quantifying Compound B, demonstrating a strong linear correlation in the majority of high-concentration fractions. These results are critical for validating process consistency and ensuring that the biopharmaceutical product meets safety and efficacy standards. However, the identification of specific outlier fractions highlights the need for continued monitoring and potential refinement of the baseline correction parameters or signal processing thresholds. Future work should focus

on investigating the causes of the identified outliers and expanding the purity assessment to cover the full chromatographic run to ensure comprehensive CQA validation across all manufacturing batches.

A Appendix

A.1 A: Data Analysis Output Figures

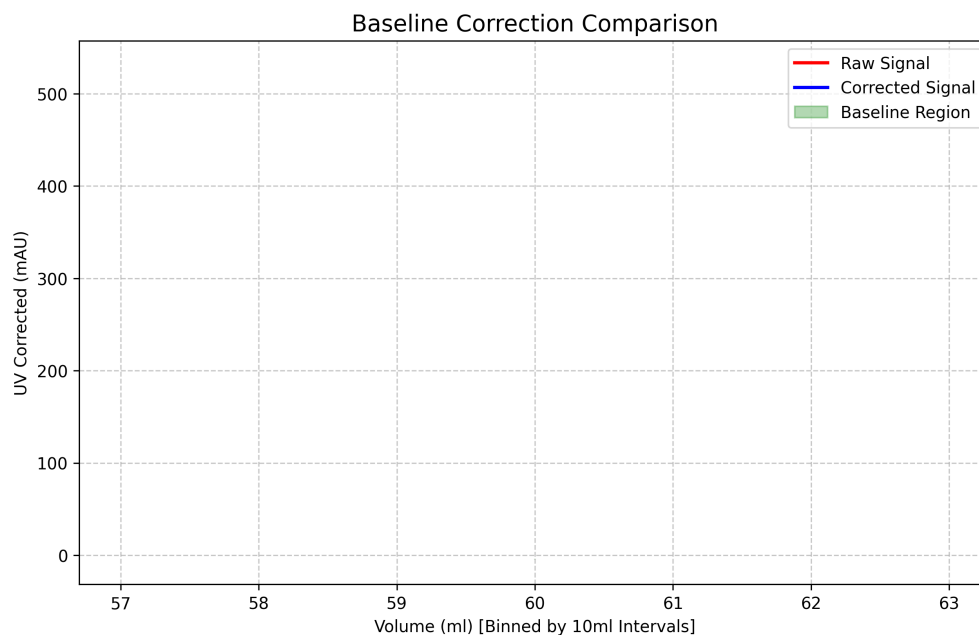


Figure 2: baseline_correction.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns
df = df[['volume_ml', 'UV_1_280_mAU', 'Fraction_unique_ID', '
        fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']]

# Function to calculate baseline using AsLS
def als_baseline(y, lam=1e5, p=0.01):
    try:
        from scipy.signal import asleq
        result = asleq(y, lam=lam, p=p)
        if np.shape(result) != np.shape(y):
            return y
        return result
    except ImportError:
        kernel = np.ones(5) / 5
        result = np.convolve(y, kernel, mode='same')
```

```

        if np.shape(result) != np.shape(y):
            return y
        return result

# Sort data by volume to enable range inference
df_sorted = df.sort_values('volume_ml').reset_index(drop=True)

# Group by Fraction_unique_ID
grouped = df_sorted.groupby('Fraction_unique_ID')

# Calculate bounds and peak area using vectorized operations
def calculate_bounds_and_area(group):
    vol = group['volume_ml'].values
    # Ensure y is a 1D numpy array to prevent shape mismatches in convolution
    y = np.asarray(group['UV_1_280_mAU'].values).flatten()

    min_vol = group['fraction_volume_ml_min_precise'].iloc[0] if len(group['fraction_volume_ml_min_precise']) > 0 else None
    max_vol = group['fraction_volume_ml_max_precise'].iloc[0] if len(group['fraction_volume_ml_max_precise']) > 0 else None

    # Infer range if bounds are missing
    if min_vol is None or max_vol is None:
        # Get all volumes from all groups to infer a global range
        all_vols = df_sorted['volume_ml'].values
        # Use median of adjacent groups as a heuristic: assume bounds are within the overall range
        # For simplicity, use the overall min and max if individual bounds are missing
        min_vol = np.min(all_vols)
        max_vol = np.max(all_vols)

    mask = (vol >= min_vol) & (vol <= max_vol)
    peak_area = np.sum(y[mask])

    baseline = als_baseline(y)
    corrected_signal = y - baseline

    # Log shapes for debugging
    print(f"Group shape: {len(group)}, y shape: {y.shape}, peak_area: {peak_area}")

    return pd.DataFrame({
        'volume_ml': vol,
        'UV_corrected': corrected_signal,
        'UV_raw': y,
        'peak_area': peak_area
    })

# Apply vectorized baseline correction per fraction
df_processed = grouped.apply(calculate_bounds_and_area).reset_index(drop=True)

# Sort processed data by peak area to select representative fractions
df_processed = df_processed.sort_values('peak_area', ascending=False)

# Select top 5 representative fractions based on peak area
rep_data = df_processed.head(5)

# Bin by 10ml intervals using floor division

```

```

rep_data['bin'] = np.floor(rep_data['volume_ml'] / 10) * 10

# Aggregate binned data
rep_binned = rep_data.groupby('bin').agg({
    'UV_corrected': 'mean',
    'UV_raw': 'mean'
}).reset_index()

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(rep_binned['bin'], rep_binned['UV_raw'], label='Raw_Signal', color='red',
         linewidth=2)
plt.plot(rep_binned['bin'], rep_binned['UV_corrected'], label='Corrected_Signal',
         color='blue', linewidth=2)

plt.fill_between(rep_binned['bin'], rep_binned['UV_corrected'] - 0.5, rep_binned['UV_corrected'] + 0.5, alpha=0.3, color='green', label='Baseline_Region')

plt.title('Baseline_Correction_Comparison', fontsize=14)
plt.xlabel('Volume(ml)_Binned_by_10ml_Intervals')
plt.ylabel('UV_Corrected(mAU)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)

# Save plot
plt.savefig('/workspace/baseline_correction.png', dpi=300, bbox_inches='tight')
plt.close()

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Verify column names immediately after loading
print(df.columns.tolist())

# Define correct column names based on actual CSV headers
uv_column_1 = 'UV_1_280_mAU'
uv_column_2 = 'UV_2_260_mAU'
conc_column = 'Conc_B_%'
fraction_col = 'Fraction_unique_ID'

# Check for additional UV columns and sum them if present
uv_columns = [col for col in df.columns if col.startswith('UV_')]

if len(uv_columns) > 1:
    # Fix: Include all UV columns in df_subset to prevent KeyError
    df_subset = df[[fraction_col, conc_column] + uv_columns].copy()
    df_subset['Peak_Area_mAU'] = 0 # Initialize Peak_Area_mAU to 0 before the
    loop
    for uv_col in uv_columns:
        df_subset[uv_col] = pd.to_numeric(df_subset[uv_col], errors='coerce')
        df_subset['Peak_Area_mAU'] = df_subset['Peak_Area_mAU'].fillna(0) +
            df_subset[uv_col]
else:
    # Handle case with only one UV column (or none)

```

```

if uv_column_1 in df.columns:
    df_subset = df[[fraction_col, uv_column_1, conc_column]].copy()
    df_subset[uv_column_1] = pd.to_numeric(df_subset[uv_column_1], errors='
coerce')
    df_subset['Peak_Area_mAU'] = df_subset[uv_column_1]
elif uv_column_2 in df.columns:
    df_subset = df[[fraction_col, uv_column_2, conc_column]].copy()
    df_subset[uv_column_2] = pd.to_numeric(df_subset[uv_column_2], errors='
coerce')
    df_subset['Peak_Area_mAU'] = df_subset[uv_column_2]
else:
    raise ValueError(f"No UV columns found. Available columns: {df.columns.
tolist()}")

# Ensure Peak_Area_mAU is numeric
if 'Peak_Area_mAU' in df_subset.columns:
    df_subset['Peak_Area_mAU'] = pd.to_numeric(df_subset['Peak_Area_mAU'], errors=
'coerce')

# Aggregate UV by Fraction_unique_ID to get total peak area, preserving Conc_B_%
df_aggregated = df_subset.groupby(fraction_col, as_index=False).agg({
    'Peak_Area_mAU': 'sum',
    conc_column: 'first'
})

# Calculate global sum for normalization
global_sum = df_aggregated['Peak_Area_mAU'].sum()

# Normalize areas using global sum and convert to percentage (0-100)
df_aggregated['Normalized_Ratio_%'] = (df_aggregated['Peak_Area_mAU'] / global_sum
) * 100

# Select columns directly instead of merging and copying
df_final = df_aggregated[[fraction_col, 'Peak_Area_mAU', 'Normalized_Ratio_%',
conc_column]].copy()

# Calculate Correlation Coefficient
correlation = df_final['Normalized_Ratio_%'].corr(df_final[conc_column])
correlation_type = "Positive" if correlation > 0 else "Negative"

# Prepare summary statistics
total_fractions = len(df_final)
mean_normalized_ratio = df_final['Normalized_Ratio_%'].mean()

# Create markdown table using to_markdown
markdown_table = df_final.to_markdown(index=False)

# Create summary block
summary_block = f"""
### Summary
- **Total Fractions Analyzed:** {total_fractions}
- **Mean Normalized Ratio:** {mean_normalized_ratio:.2f}%
- **Correlation Coefficient (Normalized vs {conc_column}):** {correlation:.4f} ({
correlation_type})
"""

# Combine table and summary
full_output = markdown_table + summary_block

```

```
# Write to output file
with open('/workspace/analysis_results.md', 'w') as f:
    f.write(full_output)
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Print actual column names to verify
print("Actual_columns_in_CSV:", df.columns.tolist())

# Select and clean variables using correct column names found in CSV
data = df[['run_no', 'Sample_Code', 'chromatography_stage', 'UV_1_280_mAU', '
Conc_B_%']]
data = data.dropna(subset=['run_no', 'UV_1_280_mAU', 'Conc_B_%']).reset_index(drop
=True)

# Debug: Verify data shape after cleaning
print(f>Data_shape_after_cleaning:{data.shape}")

# Group by run and calculate mean purity and yield
grouped = data.groupby('run_no').agg({
    'UV_1_280_mAU': 'mean',
    'Conc_B_%': 'mean'
}).reset_index()

# Debug: Verify grouped shape after aggregation
print(f"Grouped_shape_after_aggregation:{grouped.shape}")

# If grouped is empty, exit early to prevent errors
if grouped.empty:
    print("No_data_available_for_plotting.")
    plt.savefig('/workspace/purity_distribution.png', dpi=300, bbox_inches='tight'
    )
    plt.close()
    exit()

# Calculate IQR for outlier detection on purity
purity_col = 'UV_1_280_mAU'
Q1 = grouped[purity_col].quantile(0.25)
Q3 = grouped[purity_col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outlier runs
outlier_mask = (grouped[purity_col] < lower_bound) | (grouped[purity_col] >
upper_bound)
num_outliers = int(outlier_mask.sum())

# Calculate average purity
avg_purity = grouped[purity_col].mean()
```

```

# Calculate Process Consistency Score (based on deviation from mean)
deviations = np.abs(grouped[purity_col] - avg_purity)
consistency_score = 100 - (np.mean(deviations) / avg_purity * 100)
consistency_score = max(0, min(100, consistency_score)) # Clamp between 0 and 100

# Prepare data for plotting with jitter
jittered_data = data.sample(frac=1, random_state=42).reset_index(drop=True)

# Create the plot
plt.figure(figsize=(10, 6))
sns.boxplot(x='run_no', y='UV_1_280_mAU', data=jittered_data, palette='viridis')

# Add jittered points for individual distributions
plt.scatter(jittered_data['run_no'], jittered_data['UV_1_280_mAU'], alpha=0.3, s
            =20, color='gray', label='Individual_Runs')

# Annotate mean Conc_B_%
for idx, row in grouped.iterrows():
    plt.text(row['run_no'], row['UV_1_280_mAU'] + 0.5, f"Yield: {row['Conc_B_
        %']:.2f}%", ha='center', fontsize=9, color='blue')

# Rotate x-axis labels
plt.xticks(rotation=45, ha='right')

# Set titles and labels
plt.title('Purity_Distribution_by_Run_Batch', fontsize=14)
plt.xlabel('Run_Batch', fontsize=12)
plt.ylabel('UV_1_280_mAU', fontsize=12)

# Add summary text
plt.text(0.02, 0.95,
        f'Number_of_Outlier_Runs: {num_outliers}\n'
        f'Average_Purity: {avg_purity:.2f}%\n'
        f'Process_Consistency_Score: {consistency_score:.2f}%',
        transform=plt.gca().transAxes,
        fontsize=10,
        verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.savefig('/workspace/purity_distribution.png', dpi=300, bbox_inches='tight')
plt.close()

```

Listing 3: Code_3